

How to Analyse Loops for Complexity Analysis of Algorithms

Last Updated : 23 Jul, 2025



We have discussed [Asymptotic Analysis](#), [Worst, Average and Best Cases](#) and [Asymptotic Notations](#) in previous posts. In this post, an analysis of iterative programs with simple examples is discussed.

The analysis of loops for the complexity analysis of algorithms involves finding the number of operations performed by a loop as a function of the input size. This is usually done by determining the number of iterations of the loop and the number of operations performed in each iteration.

Here are the general steps to analyze loops for complexity analysis:

Determine the number of iterations of the loop. This is usually done by analyzing the loop control variables and the loop termination condition.

Determine the number of operations performed in each iteration of the loop. This can include both arithmetic operations and data access operations, such as array accesses or memory accesses.

Express the total number of operations performed by the loop as a function of the input size. This may involve using mathematical expressions or finding a closed-form expression for the number of operations performed by the loop.

Determine the order of growth of the expression for the number of operations performed by the loop. This can be done by using techniques such as big O notation or by finding the dominant term and ignoring lower-order terms.

Constant Time Complexity $O(1)$:

The time complexity of a function (or set of statements) is considered as $O(1)$ if it doesn't contain a loop, recursion, and call to any other non-constant time function.
i.e. set of non-recursive and non-loop statements

In computer science, $O(1)$ refers to constant time complexity, which means that the running time of an algorithm remains constant and does not depend on the size of the input. This means that the execution time of an $O(1)$ algorithm will always take

the same amount of time regardless of the input size. An example of an $O(1)$ algorithm is accessing an element in an array using an index.

Example:

- [swap\(\) function](#) has $O(1)$ time complexity.
- A loop or recursion that runs a constant number of times is also considered $O(1)$. For example, the following loop is $O(1)$.

C++ C Java C# JavaScript Python3

```
// Here c is a positive constant
for (int i = 1; i <= c; i++) {
    // some O(1) expressions
}

//This code is contributed by Kshitij
```

Linear Time Complexity $O(n)$:

The Time Complexity of a loop is considered as $O(n)$ if the loop variables are incremented/decremented by a constant amount. For example following functions have $O(n)$ time complexity. Linear time complexity, denoted as $O(n)$, is a measure of the growth of the running time of an algorithm proportional to the size of the input. In an $O(n)$ algorithm, the running time increases linearly with the size of the input. For example, searching for an element in an unsorted array or iterating through an array and performing a constant amount of work for each element would be $O(n)$ operations. In simple words, for an input of size n , the algorithm takes n steps to complete the operation.

C++ C Java C# JavaScript Python3

```
// Here c is a positive integer constant
for (int i = 1; i <= n; i = i + c) {
    // some O(1) expressions
}

for (int i = n; i > 0; i = i - c) {
    // some O(1) expressions
}
```

// This code is contributed by Kshitij

Quadratic Time Complexity $O(n^2)$:

The time complexity is defined as an algorithm whose performance is directly proportional to the squared size of the input data, as in nested loops it is equal to the number of times the innermost statement is executed. For example, the following sample loops have $O(n^2)$ time complexity

Quadratic time complexity, denoted as $O(n^2)$, refers to an algorithm whose running time increases proportional to the square of the size of the input. In other words, for an input of size n , the algorithm takes $n * n$ steps to complete the operation. An example of an $O(n^2)$ algorithm is a nested loop that iterates over the entire input for each element, performing a constant amount of work for each iteration. This results in a total of $n * n$ iterations, making the running time quadratic in the size of the input.

C++

C

Java

C#

JavaScript

Python3

```
// Here c is any positive constant
for (int i = 1; i <= n; i += c) {
    for (int j = 1; j <= n; j += c) {
        // some O(1) expressions
    }
}

for (int i = n; i > 0; i -= c) {
    for (int j = i + 1; j <= n; j += c) {
        // some O(1) expressions
    }
}

for (int i = n; i > 0; i -= c) {
    for (int j = i - 1; j > 0; j -= c) {
        // some O(1) expressions
    }
}
// This code is contributed by Kshitij
```

Example: [Selection sort](#) and [Insertion Sort](#) have $O(n^2)$ time complexity.

Logarithmic Time Complexity $O(\log n)$:

The time Complexity of a loop is considered as $O(\log n)$ if the loop variables are divided/multiplied by a constant amount. And also for recursive calls in the recursive function, the Time Complexity is considered as $O(\log n)$.

C++CJavaC#JavaScriptPython3

```
for (int i = 1; i <= n; i *= c) {
    // some O(1) expressions
}
for (int i = n; i > 0; i /= c) {
    // some O(1) expressions
}

// This code is contributed by Kshitij
```

C++CJavaC#JavaScriptPython3

```
// Recursive function
void recurse(int n)
{
    if (n <= 0)
        return;
    else {
        // some O(1) expressions
    }
    recurse(n/c);
    // Here c is positive integer constant greater than 1
}

// This code is contributed by Kshitij
```

Example: [Binary Search\(refer iterative implementation\)](#) has $O(\log n)$ time complexity.

Logarithmic Time Complexity $O(\log \log n)$:

The Time Complexity of a loop is considered as $O(\log \log n)$ if the loop variables are reduced/increased exponentially by a constant amount.

C++CJavaC#JavaScriptPython3

```
// Here c is a constant greater than 1
```

```
// Here c is a constant greater than 1
for (int i = 2; i <= n; i = pow(i, c)) {
    // some O(1) expressions
}
// Here fun() is sqrt or cuberoot or any other constant root
for (int i = n; i > 1; i = fun(i)) {
    // some O(1) expressions
}

//This code is contributed by Kshitij
```

See [this](#) for mathematical details.

How to combine the time complexities of consecutive loops?

When there are consecutive loops, we calculate time complexity as a sum of the time complexities of individual loops.

To combine the time complexities of consecutive loops, you need to consider the number of iterations performed by each loop and the amount of work performed in each iteration. The total time complexity of the algorithm can be calculated by multiplying the number of iterations of each loop by the time complexity of each iteration and taking the maximum of all possible combinations.

For example, consider the following code:

```
for i in range(n):
    for j in range(m):
        # some constant time operation
```

Here, the outer loop performs n iterations, and the inner loop performs m iterations for each iteration of the outer loop. So, the total number of iterations performed by the inner loop is $n * m$, and the total time complexity is $O(n * m)$.

In another example, consider the following code:

```
for i in range(n):
    for j in range(i):
        # some constant time operation
```

Here, the outer loop performs n iterations, and the inner loop performs i iterations for each iteration of the outer loop, where i is the current iteration count of the outer loop. The total number of iterations performed by the inner loop can be calculated by summing the number of iterations performed in each iteration of the outer loop, which is given by the formula $\sum(i)$ from $i=1$ to n , which is equal to $n * (n + 1) / 2$. Hence, the total time complex

C++

C

Java

C#

JavaScript

Python3

```
//Here c is any positive constant
for (int i = 1; i <= m; i += c) {
    // some O(1) expressions
}
for (int i = 1; i <= n; i += c) {
    // some O(1) expressions
}

// Time complexity of above code is O(m) + O(n) which is O(m + n)
// If m == n, the time complexity becomes O(2n) which is O(n).

//This code is contributed by Kshitij
```

How to calculate time complexity when there are many if, else statements inside loops?

As discussed [here](#), the worst-case time complexity is the most useful among best, average and worst. Therefore we need to consider the worst case. We evaluate the situation when values in if-else conditions cause a maximum number of statements to be executed.

For example, consider the [linear search function](#) where we consider the case when an element is present at the end or not present at all.

When the code is too complex to consider all if-else cases, we can get an upper bound by ignoring if-else and other complex control statements.

How to calculate the time complexity of recursive functions?

The time complexity of a recursive function can be written as a mathematical recurrence relation. To calculate time complexity, we must know how to solve

recurrences. We will soon be discussing recurrence-solving techniques as a separate post.

Algorithms Cheat Sheet:

Algorithm	Best Case	Average Case	Worst Case
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Tree Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Radix Sort	$O(dn)$	$O(dn)$	$O(dn)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Bucket Sort	$O(n+k)$	$O(n+k)$	$O(n^2)$
Counting Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$

[Quiz on Analysis of Algorithms](#)

For more details, please refer: [Design and Analysis of Algorithms](#).

Please write comments if you find anything incorrect, or you want to share more information about the topic discussed above.

Analysis of Common Loops

[Visit Course ↗](#)

 Comment

More info 

Advertise with us

Explore

DSA Fundamentals



Data Structures



Algorithms



Advanced



Interview Preparation



Practice Problem



Do Not Sell or Share My Personal Information